



**Capstone Project – Python Data
Structures and Types**
Practical Skills Module 2 Capstone Project

[Visit our website](#)

Capstone Overview

This capstone project forms part of your **Practical Skills Module 2** and is designed to assess your ability to apply Python data types and data structures in practical programming tasks.

In this capstone, you will complete four practical tasks that integrate the key topics covered in this module. You will work with strings, numbers, floats, Boolean values, lists, tuples, dictionaries, sets, and arrays to store, process, update, and organize data in Python.

Each task is based on a real-world scenario and requires you to apply multiple concepts together, rather than in isolation. You will be expected to write clear, functional Python programs that produce accurate and well-formatted outputs.

By the end of this capstone, you will have developed a set of Python files that demonstrate your ability to work confidently with core data types and data structures, and to apply them appropriately in different contexts.

You will show that you can:

- create and modify strings in Python
- work with integers, floats, and Boolean values
- perform calculations and format numeric output
- use lists, tuples, dictionaries, sets, and arrays appropriately
- access, update, remove, combine, and analyze stored data
- write clear Python programs that produce the expected results

This capstone focuses on your ability to apply Python data structures and types correctly, follow instructions carefully, and present your work in a structured and readable format.

Practical Module Scope

This capstone integrates the following practical skills:

- **PM-02-PS01:** Create, format, modify, and delete strings in Python
- **PM-02-PS02:** Use numbers in Python
- **PM-02-PS03:** Create a float in Python
- **PM-02-PS04:** Use double data types in Python

- **PM-02-PS05:** Declare a Boolean
- **PM-02-PS06:** Use Python lists to store multiple items in a single variable
- **PM-02-PS07:** Use Python tuples to store multiple items in a single variable
- **PM-02-PS08:** Use dictionaries in Python to store data values
- **PM-02-PS09:** Use Python sets to store multiple items in a single variable
- **PM-02-PS010:** Create arrays in Python

Capstone Learning Outcomes

By completing this capstone, you will be able to:

- create, format, modify, and delete strings in Python
- use integers, floats, and large numbers in calculations
- check and convert data types appropriately
- declare and evaluate Boolean values using logical conditions
- create and manipulate lists, including adding, updating, removing, and combining elements
- pack, unpack, compare, slice, and use tuples effectively
- create, access, update, merge, and manage dictionaries
- create, update, count, and combine sets
- create, access, insert, modify, remove, search, reverse, count, and traverse arrays
- write Python programs that use appropriate data structures to solve practical problems

Instructions

You are required to complete all four practical tasks in this capstone. Each task must be completed in a separate Python file using the specified filenames.

Follow these instructions carefully:

- Read each scenario and ensure you understand what the program is required to do before you begin coding.

- Create one Python file per task and use the exact filenames provided in the task instructions.
- Write code that is clear, well-structured, and easy to read. Use meaningful variable names and consistent formatting.
- Ensure that your program produces clear and labeled output for each step.
- Test your program to confirm that it runs without errors and produces the expected results.
- Apply the correct Python data types and data structures as required in each task.
- Follow all steps within each task. Do not skip any required functionality.



Practical tasks

Practical Task 1: Course Registration and Eligibility Checker

You are developing a Python program for a training provider that manages course registrations. The system must store student and course details, calculate registration costs, and determine whether a student is eligible to complete registration. Create a Python file named **course_registration.py**.

Create a Python program that:

- Stores a student's name, course title, and course code as string variables
- Combines this information into a clear registration message and displays it
 - For example: `Welcome, Alice. You have selected Introduction to Python.`
- Updates the course title to include the course code and displays the revised message
 - for example: `CS101: Introduction to Python`
- Includes examples of strings written with single quotes, double quotes, and triple quotes
- Stores values such as course fee, number of modules, or registration quantity using integers and floats

- Performs calculations such as total cost, discount, and final amount payable
- Displays each result with a clear label
- Uses `input()` to capture a numeric value, such as an additional fee or deposit amount
- Converts the input to a float and formats the result clearly
- Rounds selected float values where appropriate
- Uses Boolean variables such as `is_registered` or `has_paid_deposit`
- Compares values and combines conditions using logical operators such as `and`, `or`, or `not`
- Defines a function that returns `True` or `False` to indicate whether the student is eligible to register
- Displays the final eligibility result in a clear and readable way.

Practical Task 2: Student Performance Manager

A school requires a Python program to manage student marks and performance records. The system must store marks, calculate averages, organize student data, and allow updates to records. Create a Python file named **`student_performance_manager.py`**.

Create a Python program that:

- Creates a list of at least five student marks
- Displays the first and last values in the list
- Uses slicing to display a smaller section of the list, such as the first three marks
- Adds a new mark to the list and updates one existing mark
- Removes marks using methods such as `del`, `pop()`, and `remove()`
- Displays the list after each change so that the updates are clear
- Calculates the average mark using appropriate Python functions
- Displays the average with a clear label and suitable formatting
- Creates tuples to represent student records, such as `(name, subject, mark)`
- Unpacks tuple values into separate variables and displays them with labels

- Creates multiple tuples and sorts them using one selected value, such as the mark
- Creates a dictionary to store student records using a unique key such as a student ID
- Accesses and displays the details of one selected student
- Adds a new record, updates an existing record, and removes one record
- Checks whether a key exists before changing the dictionary
- Merges a second dictionary into the original dictionary
- Displays the final dictionary clearly

Practical Task 3: Library Collection Organizer

You are developing a Python tool for a library to manage book genres and collections. The system must track available genres, organize books, and combine collections when new books are added. Create a Python file named **library_collection_organizer.py**.

Create a Python program that:

- Creates two sets to store genre categories, such as fiction and non-fiction
- Adds a new genre to one set and removes an existing genre from another set
- Displays the updated sets clearly
- Uses the `len()` function to count the number of genres in each set
- Combines the two sets into one complete collection using an appropriate set method
- Creates a list of books for one genre and sorts the list alphabetically
 - for example: ["The Hobbit", "Harry Potter", "Game of Thrones"]
- Displays the sorted list clearly
- Creates tuples to represent book information, such as (title, genre)
- Uses these tuples as keys in a dictionary
- Creates a dictionary that stores additional information about each book, such as quantity or availability

- Removes one record from the dictionary
- Displays the updated dictionary and clearly shows that the selected record has been removed

Practical Task 4: Store Inventory Tracker

You are building a Python program for an electronics store to manage product inventory. The system must store product categories, update stock information, and allow efficient searching and analysis of inventory data. Create a Python file named **store_inventory_tracker.py**.

Create a Python program that:

- Creates collections, such as arrays or lists, to store products in categories like laptops and phones
 - for example: ["Dell", "HP", "Lenovo", "Apple"]
- Accesses and displays specific elements from the collection, such as the first or last item
- Inserts a new product at a specific position in the collection
- Modifies an existing product name
- Displays the updated collection after each change
- Removes elements using methods such as `pop()` and index-based deletion
- Displays the collection after each removal so that the changes are easy to follow
- Searches for a specific product and displays its index in the collection
- Reverses the order of the collection and displays the result
- Counts how many times a selected value appears in the collection
- Uses a loop to traverse the collection and display each item with a label
 - for example: Laptop: Dell
- Ensures that all outputs are clearly labeled and easy to read

Submission Requirements

You must submit the following Python files:

- `course_registration.py`
- `student_performance_manager.py`
- `library_collection_organizer.py`
- `store_inventory_tracker.py`



Take note

Important: Be sure to upload all files required for the task submission inside your task folder and then click "Request review" on your dashboard.



Share your thoughts

Please take some time to complete this short feedback **form** to help us ensure we provide you with the best possible learning experience.
